

An Intuitive Guide to Dynamic Programming: Solving the Coin Change Problem

Introduction: From Greedy Mistakes to Smart Solutions

Dynamic programming might sound complex, but its core idea is refreshingly simple: **we solve big problems by smartly solving smaller ones first**. This approach allows us to tackle challenges that seem overwhelmingly difficult by breaking them down into manageable pieces and building up to a final, guaranteed-best solution.

Many of us first learn about algorithms through **greedy** approaches, which make the choice that seems best at each moment. While fast and simple, this strategy can sometimes lead to suboptimal results. Dynamic programming offers a more powerful and flexible technique that ensures we find the absolute best answer every time.

Throughout this guide, we'll use the classic "**optimal change**" problem as a running example. Our goal is to design an algorithm that can find the minimum number of coins needed to make change for any amount, regardless of the coin denominations.

A Note on the Name: The term "dynamic programming" was coined by mathematician Richard Bellman in the 1950s. It's an old term that has less to do with modern computer programming and more to do with a mathematical technique of storing intermediate results in a table—a process called *tabulation*.

To see why we need a new strategy, let's first look at a simple case where a greedy approach falls short.

1. The Challenge: Making Optimal Change

Let's clearly define our problem.

Problem Statement: Given a set of coin denominations (e.g., 10, 6, and 1 cent) and a target amount (e.g., 12 cents), what is the minimum number of coins required to make change?

The Greedy Failure

The greedy approach is to always take the largest coin possible without exceeding the remaining amount. Let's try this for **12 cents** using denominations {10, 6, 1}:

1. Take a **10-cent** coin. The remaining amount is 2 cents.
2. The next largest coin (6) is too big. Take a **1-cent** coin. The remaining amount is 1 cent.
3. Take another **1-cent** coin. The remaining amount is 0 cents.

The greedy algorithm uses **3 coins** (10, 1, 1).

The "So What?" Insight

Is this the best we can do? **No.**

The optimal solution is to use two **6-cent** coins. This single example proves that the choice that seems best in the moment (taking the 10-cent coin) doesn't always lead to the best overall outcome. Our challenge isn't just to find this answer, but to build a system that *discovers* it automatically by exploring options more intelligently than a simple greedy choice.

So, how can we find the optimal solution of "two 6-cent coins" every time? The secret lies in breaking the problem into smaller, manageable pieces.

2. Breaking Down the Problem: Subproblems and Tabulation

The main strategy in dynamic programming is to solve simpler versions of the main problem first. What makes the coin change problem simpler? There are two main ways:

- **A Smaller Amount:** Finding change for 2 cents is easier than for 12 cents.
- **Fewer Coins:** Finding change using only pennies is easier than using a mix of pennies, 6-cent, and 10-cent coins.

Defining the Table

We will use a table (an array) to store the solutions to these simpler subproblems. This powerful technique of storing and reusing intermediate results is called **tabulation**.

Each cell in our table, $m[i][j]$, will represent the answer to a very specific subproblem:

The minimum number of coins to make change for amount j using only the coins from i down to n .

In our example, $m[2][j]$ would be the minimum coins for amount j using only $\{6, 1\}$, while $m[3][j]$ would use only $\{1\}$.

Illustrating the Simplest Case

Let's start by solving the absolute easiest subproblem: making change using only pennies ($v[3]=1$). This will be the bottom row of our table. The solution is trivial: to make change for an amount j , you need j pennies.

Amount j	0	1	2	3	4	5	6	7	8	9	10	11	12
Coins: {1}	0	1	2	3	4	5	6	7	8	9	10	11	12

With this simple foundation in place, we can now build upon it to solve more complex subproblems, one coin at a time.

3. The Recurrence: Finding the Pattern

As we add a new, larger coin denomination (like the 6-cent coin), we face a simple choice for any given amount j :

- **Case 1: We DON'T use the new coin.** In this case, the best solution is the one we already found using only the smaller coins. We can just look this answer up in the row below us in the table.
- **Case 2: We DO use the new coin.** In this case, the total number of coins will be 1 (for the new coin we just used) plus the optimal number of coins for the *remaining amount*. We find this by looking up the solution for the remaining amount ($j - v[i]$) *in the very same row*, since we are still allowed to use the new coin again.

This logic gives us a rule, or a **recurrence relation**, to fill out every cell in our table.

The Recurrence Relation

$$m[i][j] = \min\{ m[i+1][j], 1 + m[i][j - v[i]] \}$$

Demystifying the Formula

Let's break down what each part means in plain English:

- $m[i+1][j]$: This represents the cost if we **don't** use coin $v[i]$. It's the best solution we already found in the row below (which used a smaller set of coins).
- $1 + m[i][j - v[i]]$: This represents the cost if we **do** use coin $v[i]$. It's 1 (for the coin we're using now) plus the optimal solution for the amount that's left over ($j - v[i]$).
- $\min\{\dots\}$: This simply means we compare the results of the two cases and pick the one that uses fewer coins.

Now that we have our rule, let's use it to fill out the entire table and find our final answer.

4. Putting It All Together: The Full Solution

By applying our recurrence relation, we can systematically build our table from the bottom up, from the simplest subproblems to the most complex. Here is the complete table. Notice how the bottom row matches the simple case we started with, and each new row builds upon the optimal answers from the row below it.

Walkthrough (Adding the 6-cent coin)

Let's calculate the middle row, which allows the use of {6, 1} coins. We'll focus on finding the optimal change for 12 cents. The table below highlights the key values used in this calculation.

Coins i	0	1	2	3	4	5	6	7	8	9	10	11	12
{10, 6, 1}	0	1	2	3	4	5	1	2	3	4	1	2	2
{6, 1}	0	1	2	3	4	5	1	2	3	4	5	6	2
{1}	0	1	2	3	4	5	6	7	8	9	10	11	12

At amount $j = 12$, our choice is:

```
min( Cost if we DON'T use a 6¢, Cost if we DO use a 6¢ ) min( m[row  
below][12], 1 + m[same row][12 - 6] ) min( 12, 1 + m[same row][6] )
```

To solve this, we need $m[\text{same row}][6]$. Our algorithm would have already calculated this. At $j=6$, the choice was $\min(m[\text{row below}][6], 1 + m[\text{same row}][0])$, which is $\min(6, 1 + 0) = 1$.

Now we can finish our calculation for 12 cents. We look up the value for 6 cents, which our table has already proven is optimally solved with **1 coin**.

```
min( 12, 1 + 1 ) = 2
```

The minimum is 2. This is the key insight: two 6-cent coins are better than twelve 1-cent coins. We record 2 in our table.

Walkthrough (Adding the 10-cent coin)

Finally, let's calculate the top row for {10, 6, 1} and find our final answer for 12 cents. The table below highlights this final calculation.

Coins i	0	1	2	3	4	5	6	7	8	9	10	11	12
{10, 6, 1}	0	1	2	3	4	5	1	2	3	4	1	2	2
{6, 1}	0	1	2	3	4	5	1	2	3	4	5	6	2
{1}	0	1	2	3	4	5	6	7	8	9	10	11	12

At amount $j = 12$, our choice is:

```
min( Cost if we DON'T use a 10¢, Cost if we DO use a 10¢ ) min( m[row
below][12], 1 + m[same row][12 - 10] ) min( 2, 1 + m[top row][2] )
```

We look up the values from our table. The solution from the row below is a *guaranteed optimal solution* for the $\{6, 1\}$ coin set, which we found to be 2. The other option requires us to find the value for $m[\text{top row}][2]$, which the table already tells us is 2.

```
min( 2, 1 + 2 ) min( 2, 3 ) = 2
```

The algorithm chooses 2. This final result correctly ignores the suboptimal greedy choice (3 coins) because it was able to compare it against a better solution built from the optimal answers to smaller subproblems.

By systematically building a table of solutions to smaller problems, we were able to guarantee the optimal answer for our original, larger problem.

5. What We Learned: The Principles of Dynamic Programming

Now that we've worked through an example, we can formalize the two core properties of problems that are well-suited for dynamic programming.

Concept	Description	How It Applies to Coin Change
Optimal Substructure	An optimal solution to the main problem contains optimal solutions to smaller subproblems.	The optimal way to make change for 12 cents (e.g., using a 6-cent coin) relies on knowing the optimal way to make change for the remaining 6 cents.
Overlapping Subproblems	The algorithm solves the same small subproblems over and over again.	For instance, our final calculation for 12 cents required the optimal solution for 2 cents ($1 + m[\text{top row}][2]$). A purely recursive algorithm might recompute this from scratch. Our table, however, had the answer '2' stored and ready, preventing redundant work. This is the power of solving overlapping subproblems just once and storing the result.

Dynamic programming is a powerful tool for optimization, and by understanding these core principles, you can start to recognize other problems where it can be applied.